

UNIVERSITY OF LIVERPOOL

Department of Computer Science

**Systematic Evaluation of Prompt Engineering Techniques
for Large Language Models:
A Multi-Task Empirical Framework**

A dissertation submitted in partial fulfilment of the requirements
for the degree of Master of Science in Computer Science

Varun Kumar

Student ID: [Insert Student ID]

Supervisor: Achilleas Koufonikos

August 2026

Dedication

To my family, for their patience and unwavering support throughout this project.

And to anyone who has ever spent an hour rephrasing a question
to get a better answer - you have been doing prompt engineering all along.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Achilleas Koufonikos, for his guidance, encouragement, and thoughtful feedback throughout this project. His advice on the experimental design and statistical approach was invaluable, and his willingness to discuss ideas at short notice made a real difference to the quality of the work.

I am also grateful to the University of Liverpool for providing access to the resources and infrastructure that made this project possible, and to Anthropic for their well-documented Claude API, without which the experiments could not have been conducted.

A special thank you to friends and fellow students on the MSc programme who provided informal feedback on the dashboard and listened patiently while I explained the difference between zero-shot and chain-of-thought prompting for what was probably the fifth time.

Finally, and most importantly, thank you to my family for their support, patience, and the occasional timely reminder that I had not eaten or slept.

UNIVERSITY OF LIVERPOOL

Department of Computer Science

**Systematic Evaluation of Prompt Engineering Techniques
for Large Language Models:
A Multi-Task Empirical Framework**

MSc Computer Science Dissertation

August 2026

Abstract

This dissertation presents the Prompt Engineering Evaluation Framework (PEEF), a Python toolkit designed to run controlled, reproducible experiments comparing four prompting strategies - zero-shot, few-shot, chain-of-thought (CoT), and zero-shot CoT - across three natural language processing task domains: factual question answering, text summarisation, and mathematical reasoning. All experiments are conducted via the Anthropic Claude API (claude-haiku-4-5-20251001 model) on established public benchmarks: TriviaQA, CNN/DailyMail, and GSM8K.

The core finding is that no single technique is universally best, and that the optimal strategy is strongly task-dependent. Few-shot (4-shot) prompting leads on QA at 70.6% exact match; with 500 examples the margin over zero-shot (67.4%) reaches statistical significance ($p = 0.018$), though the effect size is negligible ($d = 0.07$), confirming that the practical difference is small. Zero-shot performs best for summarisation with a mean ROUGE-L of 24.0%, significantly outperforming CoT (20.3%, $p < 0.001$, medium effect). CoT dominates mathematical reasoning at 90.5% exact match, a 27-percentage-point improvement over zero-shot (63.4%) that is statistically significant with a medium effect size (Cohen's $d = 0.68$). Zero-shot CoT is counterproductive for factual QA but provides a strong intermediate option for reasoning tasks at 82.8%.

Beyond the static comparison, the framework introduces two novel contributions. The Reflexive Prompt Optimizer (Module 5) implements an automated evaluate-reflect-mutate-select loop in which the model critiques its own failure cases and proposes improved templates, gated by a paired bootstrap significance test - a statistical guard against dev-set noise that prior automated prompting work has lacked. The Adaptive Cascade (Module 6) routes each question individually rather than applying a fixed technique to all questions, using self-consistency agreement among cheap zero-shot samples as a confidence signal, and escalating to CoT only when confidence falls below a tunable threshold. The cost-efficiency analysis demonstrates that this approach can approximate CoT-level accuracy at significantly lower token cost.

The project is accompanied by an interactive Streamlit dashboard deployed to Streamlit Cloud, providing a public interface for exploring results, statistical test outputs, and cross-task comparisons. The complete codebase, experiment configurations, and results are available on GitHub.

Statement of Ethical Compliance

Data Category: A0

This project does not collect personal data and does not involve human participants in any experimental activity. All datasets used - TriviaQA, CNN/DailyMail, and GSM8K - are publicly available benchmarks released under open licences (Apache 2.0 and MIT) and contain no personal or identifiable information. I confirm that this project has been carried out in full accordance with the University of Liverpool's ethical guidance for computing projects, as described more fully in Section 7 of this dissertation.

Table of Contents

Acknowledgements	3
Abstract	5
Statement of Ethical Compliance	6
Table of Contents	7
List of Figures	8
1 Introduction and Background	9
1.1 Motivation	9
1.2 Research Questions	11
1.3 Aims and Requirements	11
1.4 Project Overview	12
1.5 Dissertation Structure	13
2 Design and Implementation	14
2.1 System Architecture	14
2.2 Prompt Design	15
2.3 Experiment Runner	18
2.4 Evaluation Framework	20
2.5 Statistical Analysis Engine	21
2.6 Reflexive Prompt Optimizer (Novel)	22
2.7 Adaptive Cascade (Novel)	25
2.8 Streamlit Dashboard	27
3 Testing and Evaluation	29
3.1 Unit Testing	29
3.2 Question Answering Results	30
3.3 Summarisation Results	32
3.4 Mathematical Reasoning Results	34
3.5 Cross-Task Analysis and Cost-Efficiency	36
4 Project Ethics	38
5 Conclusion and Future Work	39
6 BCS Project Criteria and Self-Reflection	42
References	45

List of Figures

Figure 1: PEEF system architecture showing the six-module pipeline	14
Figure 2: Per-task performance results across all four prompting techniques	31
Figure 3: Combined overview chart showing accuracy and ROUGE-L by task and technique	33
Figure 4: Effect size summary (Cohen's d) for all significant pairwise comparisons	35
Figure 5: Streamlit dashboard wireframe designed during the planning phase	28

List of Tables

Table 1: Project requirements (R1-R8)	12
Table 2: Dataset characteristics for the three experimental task domains	13
Table 3: YAML configuration file for the QA zero-shot experiment	19
Table 4: Question answering results (exact match, n = 200)	30
Table 5: Summarisation results (ROUGE-L F-score, n = 200)	32
Table 6: Mathematical reasoning results (exact match, n = 200)	34
Table 7: Best technique per task - summary	36
Table 8: Statistical significance of pairwise comparisons (Wilcoxon, alpha = 0.05)	37

1 Introduction and Background

1.1 Motivation

There is a particular kind of frustration that anyone who has worked with a large language model will recognise: asking the same question twice and getting very different answers, or watching the model fail at something straightforward and then succeed at something far harder. Much of the time, the difference comes down not to what you asked but how you asked it. The exact wording of the input - the prompt - turns out to be one of the most consequential variables in applied AI work.

This sensitivity to phrasing gave rise to prompt engineering: the deliberate, systematic design of inputs to language models to achieve better outputs. Over the past four years, several techniques have emerged that reliably improve model performance. Zero-shot prompting presents the model with only a task instruction and no examples, relying entirely on knowledge absorbed during pre-training. Few-shot prompting provides a small number of worked examples in the prompt itself, giving the model a pattern to follow without any gradient updates. Chain-of-thought (CoT) prompting goes further, asking the model to reason through a problem step by step before committing to a final answer.

Brown et al. (2020) established the foundational result that large models can perform tasks from examples provided purely in context - what they called in-context learning - without any fine-tuning. Their demonstration of zero-shot and few-shot prompting on GPT-3 defined the vocabulary the field still uses today. Wei et al. (2022) then showed that including a reasoning chain in the examples, rather than just input-output pairs, produced striking improvements on arithmetic and symbolic reasoning tasks that pure few-shot prompting could not match. Separately, Kojima et al. (2022) demonstrated something simpler and, in some ways, more surprising: that adding just six words - "Let's think step by step" - to a zero-shot prompt could unlock reasoning capabilities without any examples at all. Their technique, now called zero-shot CoT, forms the fourth condition tested in this project.

What the existing literature has not done well is compare these techniques across different task types while simultaneously measuring the token cost they impose. Academic papers typically evaluate a single technique on a single task type, making it difficult to draw cross-domain conclusions. More importantly, they rarely discuss cost: using a language model through an API charges by the token, and a technique that improves accuracy by five percentage points while tripling token consumption is not always the right choice. A developer building a production QA system has different priorities from a researcher benchmarking arithmetic reasoning, and neither group is well-served by advice that ignores cost entirely.

This project addresses that gap. The Prompt Engineering Evaluation Framework (PEEF) is a Python toolkit that runs controlled, reproducible experiments comparing all four techniques - zero-shot, few-shot (4-shot), CoT, and zero-shot CoT - across three task domains: factual question answering, text summarisation, and mathematical reasoning. Every experiment is defined by a YAML configuration file and logged with full context, so results are always traceable. Statistical tests (Wilcoxon signed-rank) and effect sizes (Cohen's d) are computed

automatically, ensuring that conclusions are grounded in evidence rather than visual inspection of mean scores.

Beyond the static comparison, the project makes two additional contributions that none of the four static techniques alone can support. The Reflexive Prompt Optimizer asks the model to diagnose its own failure cases and proposes improved templates, gated by a statistical test that prevents chance improvements from being promoted. The Adaptive Cascade routes each question individually: it tries a cheap zero-shot probe first and only escalates to the more expensive CoT technique when the probe reveals low confidence. Both modules are designed to be testable without API access, with injectable mock functions that allow the full test suite to run offline.

The result is not just an experimental finding but a practical, reusable framework with evidence-backed guidelines for practitioners deciding how to prompt a language model for a given task type.

1.2 Research Questions

The project is structured around four research questions.

- RQ1.** How do zero-shot, few-shot, and chain-of-thought prompting compare in accuracy and output quality across different task types?
- RQ2.** Does the optimal prompting technique vary by task domain, and if so, how strongly?
- RQ3.** What prompt design principles generalise across techniques to improve LLM performance?
- RQ4.** Can prompts be improved automatically from their own failure cases, and can per-instance adaptive technique selection approximate chain-of-thought accuracy at lower token cost?

1.3 Aims and Requirements

The project has four primary aims. First, to build PEEF as a clean, modular, and extensible Python framework for running and scoring prompt engineering experiments via the Claude API. Second, to run a full comparative experiment across all four techniques and three task domains using established public benchmarks. Third, to go beyond accuracy alone and quantify the cost-efficiency of each technique, producing practical guidance for real applications. Fourth, to investigate whether prompts can improve themselves and whether adaptive per-instance routing can reduce token cost without sacrificing accuracy.

Requirements were derived from these aims through discussion with the supervisor and are listed below in priority order.

Table 1: Project requirements (R1-R8). R1-R6 are essential; R7-R8 are desirable.

ID	Requirement	Priority
R1	Zero-shot, few-shot (4-shot), CoT, and Zero-Shot CoT implemented in PEEF	Essential
R2	Experiments run across QA, Summarisation, and Reasoning tasks	Essential
R3	Correct metric applied per task: exact match, ROUGE-L, BERTScore	Essential
R4	Full result logging: prompt, model, temperature, timestamp, tokens	Essential
R5	Statistical analysis: Wilcoxon signed-rank tests + Cohen's d effect sizes	Essential
R6	Codebase fully documented with docstrings, README, and YAML configs	Essential
R7	Interactive Streamlit dashboard for result visualisation	Desirable
R8	Token cost tracking and cost-efficiency visualisation	Desirable

All eight requirements were delivered. The project additionally contributed two novel adaptive modules (Module 5: Reflexive Prompt Optimizer; Module 6: Adaptive Cascade) that were not part of the original requirements but emerged from the literature review and the desire to go beyond static comparison.

1.4 Project Overview and Datasets

All experiments use Anthropic's Claude API with the claude-haiku-4-5-20251001 model, which is a frontier-scale model capable of exhibiting the chain-of-thought reasoning behaviour documented by Wei et al. (2022). Three publicly available benchmark datasets were selected, one per task domain.

Table 2: Dataset characteristics for the three experimental task domains.

Dataset	Task	Licence	N (used)	Primary Metric
TriviaQA (Joshi et al., 2017)	Factual QA	Apache 2.0	200	Exact Match
CNN/DailyMail (See et al., 2017)	Summarisation	MIT	200	ROUGE-L + BERTScore
GSM8K (Cobbe et al., 2021)	Mathematical Reasoning	MIT	200	Exact Match

TriviaQA was chosen for QA because its questions are open-domain and test a wide range of factual knowledge, making them a fair test of what a model knows without examples. CNN/DailyMail is the standard abstractive summarisation benchmark, pairing news articles with human-written bullet-point summaries. GSM8K (Grade School Math 8K) is a dataset of grade-school arithmetic word problems requiring between two and eight reasoning steps; it is the benchmark used by Wei et al. (2022) and Kojima et al. (2022) to demonstrate CoT gains, making it directly comparable to those results.

The sample sizes of 200 per task were chosen to be large enough for reliable Wilcoxon signed-rank tests while keeping API costs manageable within the 12-week project timeline. All experiments run at temperature 0 for deterministic output.

1.5 Dissertation Structure

Section 2 describes the design and implementation of all six PEEF modules, the YAML-driven experiment pipeline, and the key engineering decisions made along the way - including the problems encountered and how they were resolved. Section 3 presents the experimental results for all three task domains, with statistical tests and a cross-task cost-efficiency analysis. Section 4 addresses project ethics. Section 5 answers the four research questions, discusses limitations, and outlines directions for future work. Section 6 addresses the six BCS project criteria and includes a critical self-reflection on the project process.

2 Design and Implementation

2.1 System Architecture

PEEF was designed around three principles. First, every experiment had to be reproducible: given the same YAML configuration file and an API key, the exact same results should be obtainable by anyone at any time. Second, the codebase had to be modular: each module is responsible for exactly one concern, with no coupling between, say, the evaluator and the prompt builder. Third, the framework had to be extensible: adding a new technique, a new dataset, or a new metric should require no changes to existing modules.

The six modules and their responsibilities are: the Prompt Builder (Module 1) constructs formatted prompts from Jinja2 templates; the Experiment Runner (Module 2) manages Claude API calls with caching and rate-limit handling; the Evaluator (Module 3) scores model responses against ground-truth references; the Analysis Engine (Module 4) computes statistics and produces charts; the Reflexive Prompt Optimizer (Module 5) evolves prompt templates automatically; and the Adaptive Cascade (Module 6) routes questions to the most appropriate technique on a per-instance basis.

Data flows through the framework in a single direction: a YAML configuration file specifies an experiment; the Prompt Builder constructs prompts from the specified dataset and technique; the Experiment Runner sends them to the API and persists the raw responses; the Evaluator scores the responses; and the Analysis Engine reads the scored results to produce statistics and visualisations. The dashboard reads from the same scored results directory. This clean separation means that any module can be replaced or upgraded without touching the others.

[Figure 1 - PEEF Architecture Diagram]

Figure 1: PEEF system architecture. The six modules communicate through a shared file-based results store, ensuring full reproducibility and separation of concerns.

2.2 Prompt Design

The design of the prompt templates was one of the most consequential decisions in the whole project. If technique A outperforms technique B, it should be because of the prompting strategy, not because A's template is better worded. To control for this, all templates were written following the same principles: they use natural English instructions, they specify the required output format explicitly, and they include only information that is specific to the technique being tested.

The Prompt Builder uses Jinja2 templates, one per task-technique combination, embedded directly in the module so they are always in sync with the code. A template is identified by a composite key (for example, "qa_zero_shot" or "reasoning_cot"), and the PromptBuilder renders it with input data from the dataset. For few-shot and CoT techniques, the examples

pool is passed as part of the context and the template inserts the first `n_shots` items automatically.

The zero-shot QA template reads as follows:

```
Answer the following question. Give only the answer, no explanation.
```

```
Question: {{ question }}
```

```
Answer:
```

The instruction "Give only the answer, no explanation" was added after observing during pilot runs that Claude without this constraint sometimes returned explanatory paragraphs. These would contain the correct answer but fail exact-match scoring because the response string did not equal the reference string. This is a good example of how prompt design affects measured results in ways that have nothing to do with the technique itself: the fix was a four-word constraint that made the output format unambiguous.

The few-shot QA template extends this by inserting `k` worked examples before the target question. Each example is an input-output pair from a fixed pool, meaning all runs for a given `n_shots` configuration see the same examples, preserving comparability across runs. The 4-shot condition uses four examples selected to cover a range of question types.

The CoT template is the most structurally different from the others. Rather than showing input-output pairs in the examples, it shows input-reasoning-output triples, with a worked reasoning chain connecting the question to its answer. For mathematical reasoning, this looks like:

```
Solve each maths problem by showing your working step by step, then state the final answer.
```

```
Problem: If a train travels at 60 mph for 2 hours, how far does it travel?
```

```
Working: Distance = speed x time = 60 x 2 = 120 miles.
```

```
Answer: 120
```

```
Problem: {{ problem }}
```

```
Working:
```

Leaving the template open at "Working:" prompts the model to continue the reasoning chain, which it then follows with "Answer:". This two-stage structure is what enables the exact-match extractor in the Evaluator to find the final numerical answer even when the response contains a long working trace. The same pattern was adapted for QA and summarisation, with task-appropriate labels.

Zero-shot CoT follows Kojima et al.'s (2022) formulation exactly, appending "Think step by step" to the zero-shot instruction. This was deliberately kept as a minimal modification rather

than a rewrite, so that any performance difference between zero-shot and zero-shot CoT is attributable to the addition of the reasoning instruction alone.

The summarisation templates required extra care because the output format varies more than for QA or reasoning. "Summarise in 2-3 sentences" was chosen as the output constraint after piloting with "Summarise concisely" (which produced summaries ranging from one to eight sentences) and "Provide a brief summary" (which sometimes produced bullet-point lists instead of prose). The 2-3 sentence constraint makes outputs comparable across techniques and more closely matches the CNN/DailyMail reference format.

2.3 Experiment Runner

The Experiment Runner handles all communication with the Claude API. Three engineering concerns drove its design: cost control, reproducibility, and resilience to rate limiting.

For cost control, the runner maintains a flat-file deduplication cache. Every completed run is identified by a deterministic hash derived from the configuration name and the example ID, and this hash is stored in a text file after each successful API call. Before sending a prompt, the runner checks whether its hash is already in the cache; if so, it skips the call entirely. This meant that when experiments were interrupted - which happened several times due to rate limiting - they could be resumed from the last completed point without any duplicate charges or data inconsistency.

The YAML configuration files define every experimental parameter. An example for the QA zero-shot experiment is shown below.

Table 3: YAML configuration file for the QA zero-shot experiment. All twelve experiments are defined in analogous files.

Parameter	Value	Purpose
name	qa_zero_shot	Unique identifier for caching and logging
task	qa	Determines which template and metric to use
technique	zero_shot	Technique identifier passed to PromptBuilder
n_shots	0	Number of in-context examples (0 for zero-shot)
dataset	trivia_qa	Source dataset name for dataset_loader utility
sample_size	500 / 300 / 1319	QA / Summarisation / Reasoning (full

Parameter	Value	Purpose
		proposed sets)
model	claude-haiku-4-5-20251001	API model identifier
temperature	0.0	Deterministic output for comparability
max_tokens	128	Response length cap for QA

Rate limiting was one of the most significant practical challenges of the project. The Claude API enforces requests-per-minute limits, and sending 200 prompts in rapid succession consistently triggered these. The runner implements exponential back-off: when a rate-limit error is received, it waits a delay that doubles on each subsequent retry (1 second, 2, 4, 8...) before retrying the call. This made overnight batch runs reliable but introduced unpredictability in run time, since a batch hitting rate limits could take twice as long as planned. The eventual solution was to run experiments in smaller groups and rely on the deduplication cache to reassemble the complete results from multiple runs.

A subtler problem was the handling of API errors that are not rate limits - network timeouts, malformed responses, and occasional model errors. The runner catches all such exceptions, writes an error record to disk with the error message and the failed run ID, and continues to the next prompt rather than aborting the batch. This meant that the results directory would sometimes contain a small number of error records that needed to be filtered out before evaluation. These were typically around 0.5-1% of all runs, and re-running them specifically via the cache miss mechanism recovered the data.

All successful results are written to disk immediately after each API call as individual JSON files containing the full prompt, the raw model response, token counts, timestamp, model version, temperature, and any metadata. This append-only design means that no completed results are ever lost, even if the process crashes mid-batch.

2.4 Evaluation Framework

The Evaluator module scores model responses against ground-truth references. The choice of metric per task was made to match what a human judge would consider a correct answer for each task type.

For question answering and mathematical reasoning, exact match is used after normalisation. Both the prediction and the reference are lowercased, punctuation is stripped using Python's string.punctuation table, and whitespace is collapsed to a single space. This handles the most common formatting differences - trailing full stops, capitalisation differences, comma-separated thousands in numbers - without introducing ambiguity.

For mathematical reasoning specifically, the evaluator applies a number extractor before normalisation. Because CoT prompts produce long working traces before the final answer, the prediction string may be hundreds of characters long. The extractor uses a regular expression to find all numbers in the string and returns the last one, on the assumption that the final numerical value is the answer. This worked reliably across the 200 GSM8K examples, though edge cases involving percentage answers required a small adjustment to handle the "%" character correctly.

For summarisation, two complementary metrics are used. ROUGE-L (Lin, 2004) measures the longest common subsequence overlap between the generated summary and the reference, capturing structural similarity. BERTScore (Zhang et al., 2020) computes cosine similarity between contextualised BERT embeddings, matching tokens semantically rather than requiring exact string overlap. This means that a summary using "automobile" where the reference says "car" scores well under BERTScore but poorly under ROUGE-L.

Using both metrics is important precisely because they disagree in interesting ways. In the results, ROUGE-L and BERTScore tell somewhat different stories about the summarisation task, and that difference is itself informative about the nature of the summaries being generated by each technique.

A bug in the evaluation direction for ROUGE-L was caught during the testing phase. The `rouge_score` library accepts (reference, prediction) as its argument order, but an early version of the Evaluator passed them in the wrong order, giving scores that were technically measuring the reverse metric (precision rather than recall, in effect). This was caught by running the evaluator against a manually computed example and comparing the output to a known correct score. Both the implementation and the corresponding unit test were corrected.

2.5 Statistical Analysis Engine

The Analysis Engine takes the collection of scored result files and produces summary statistics, pairwise significance tests, and visualisation charts. Its outputs are read both by the dissertation figures and by the Streamlit dashboard.

For significance testing, the Wilcoxon signed-rank test was chosen over a paired t-test. The reasoning is straightforward: for QA and reasoning tasks, per-instance scores are binary (0 or 1), so the distribution is clearly non-normal. The Wilcoxon test is non-parametric and makes no distributional assumptions, making it appropriate for both the binary and the continuous (ROUGE-L) score distributions. The test is applied in a paired fashion: both techniques are run on the same 200 examples, so each test compares per-instance scores for the same inputs across two conditions.

Effect sizes are reported as Cohen's d alongside every p-value. This is important because with larger sample sizes ($n=500$ for QA, $n=1,319$ for reasoning), even practically trivial differences can reach statistical significance. Cohen's d tells us whether a significant difference is large enough to matter in practice. The conventional thresholds used are $|d| < 0.2$ (negligible), 0.2-0.5 (small), 0.5-0.8 (medium), and above 0.8 (large). All confidence intervals are computed as $1.96 \times (\text{standard deviation} / \sqrt{n})$ to give approximate 95% intervals.

The statistical summary for all three tasks and all pairwise comparisons is saved as a single JSON file (03_results/analysis/statistical_summary.json) that is loaded directly by the dashboard and referenced in Section 3 of this dissertation. This ensures that the numbers in the text and in the dashboard always come from the same source.

2.6 Reflexive Prompt Optimizer (Novel Contribution)

The four static techniques compared in the main experiments represent a snapshot: each technique uses a hand-crafted template written once, at the start of the project. The templates are reasonable - they were written with care and piloted before the main runs - but they are not optimal. There is no principled reason to believe that the first reasonable formulation of a prompt is the best one.

The Reflexive Prompt Optimizer (RPO) addresses this by treating the prompt template itself as an artefact that can be improved iteratively. The optimisation loop has four phases, which repeat for a configurable number of generations or until a patience condition stops early progression.

The Evaluate phase runs the current champion template on a held-out development set and records per-item scores. This establishes a baseline that the challenger must beat. The Reflect phase presents the LLM with its worst-scoring development items and asks it to diagnose what about the template - its wording, structure, output format constraints, or missing guidance - caused those specific failures. Importantly, the critique prompt instructs the model to blame only the template, not the model or the data, keeping the feedback constructive and actionable.

The Mutate phase shows the current template, the critique, and the required Jinja2 placeholder variables to the LLM at a higher temperature (0.9), asking it to produce K diverse alternative templates that address the diagnosed failures. Diversity is explicitly enforced: the prompt instructs the model not to produce paraphrases but meaningfully different structural approaches. Each candidate is checked to ensure it contains exactly the required placeholders before being accepted.

The Select phase evaluates each candidate on the same development set and applies a one-sided paired bootstrap test to determine whether the candidate's improvement over the champion is statistically real. The test resamples the paired per-item score differences 10,000 times and computes the fraction of resamples in which the candidate does not outperform the champion. A candidate is promoted only if this p-value falls below 0.05 (the conventional alpha). This statistical gate is the key distinguishing feature of the RPO relative to prior automatic prompt engineering work.

Existing automatic prompting frameworks - notably APE (Zhou et al., 2022) and OPRO (Yang et al., 2023) - select the best-scoring candidate from a generation simply by comparing mean scores. On a development set of 50-100 items, a candidate can score higher than the champion entirely by chance: if the items happen to be those where the new wording does slightly better, the mean looks good even if there is no real improvement. The bootstrap test prevents this. Empirically, in the development runs, there were several generations where a candidate scored higher than the champion by 2-3 percentage points but failed the bootstrap test ($p > 0.1$),

indicating that the apparent improvement was within sampling noise. These candidates were correctly rejected.

Every API call, critique, candidate template, score, p-value, and promotion decision is written to a JSON audit-trail file that records the full optimization lineage. The module also tracks total token usage and estimated API cost using a cost ledger, and stops automatically if a hard token budget is exceeded. This makes the entire optimization process reproducible and transparent.

2.7 Adaptive Cascade (Novel Contribution)

The static comparison answers the question "which technique is best on average for this task?" A more useful question for a practitioner is "which technique does this specific question need?" The insight that motivates the Adaptive Cascade is that not all questions are equally hard: some are answered correctly by a fast, cheap zero-shot prompt every time, while others genuinely need the structured reasoning that CoT provides. Applying the expensive technique to every question wastes tokens on the easy ones.

The Adaptive Cascade implements a two-tier routing mechanism. For each input, Tier 1 samples k responses from a cheap zero-shot prompt at a temperature above zero (temperature = 0.7). The agreement ratio among the k answers is a self-consistency confidence estimate, adapting the idea from Wang et al. (2022): if all k samples agree, the model is confident and the majority answer is accepted directly. If the samples disagree, confidence is low and the input escalates to Tier 2, which applies CoT with a deterministic temperature-0 setting and returns its answer.

The routing logic is controlled by a single confidence threshold parameter. Setting the threshold at 1.0 means the cascade escalates to CoT unless all k samples are unanimous - this is the conservative setting that escalates frequently. Setting it at 0.0 means nothing ever escalates and the cascade becomes equivalent to pure zero-shot. Any value between these extremes defines a point on the accuracy-cost trade-off curve, and the optimal threshold depends on the application's budget constraints.

The cascade includes a threshold sweep utility that re-derives the escalation curve across any list of threshold values using only the stored Tier-1 confidence scores, without making additional API calls. This means the full Pareto frontier of accuracy versus escalation rate can be characterized from a single run of the cascade at one threshold value. For the dissertation results, the cascade was run at threshold = 0.7 with $k = 3$ samples per question on the reasoning task, where the gap between zero-shot and CoT is largest and the trade-off is therefore most meaningful.

The module accepts an injectable `call_fn` parameter, which means the entire test suite can run offline by passing a deterministic mock function that returns fixed responses. This design pattern was applied consistently across all modules that make API calls: the real API call is behind an abstraction that can always be replaced by a test double, making the test suite independent of network access or API keys.

2.8 Streamlit Dashboard

The Streamlit dashboard provides a visual interface for exploring the experimental results without requiring the user to write any code. It is implemented as a single Python file that loads from the scored results directory and the statistical summary JSON, and organises its content into five pages accessed from a sidebar: Home, Results, Charts, Statistics, and Framework.

The Home page shows three key animated KPI counters and three gauge charts highlighting the best-performing result from each task domain: Few-Shot at 70.6% on QA, Zero-Shot at 24.0% ROUGE-L on summarisation, and CoT at 90.5% on reasoning. The Results page shows interactive grouped bar charts for each task, with technique on the x-axis and score on the y-axis, generated by Plotly Express from the live JSONL files. The Charts page contains the radar/spider chart showing all four techniques across all three tasks simultaneously, and the individual gauge charts. The Statistics page displays the full Wilcoxon results table from the JSON file. The Framework page describes the six PEEF modules with expandable sections.

Two implementation problems arose during dashboard development that are worth documenting.

The first was a Plotly compatibility issue with radar chart fill colours. The initial implementation used 8-digit hexadecimal colour codes (for example, #0072B222, where the last two digits encode opacity) to produce semi-transparent fill regions on the radar traces. This rendered correctly in the local development environment but threw a `ValueError` in the Plotly version installed on Streamlit Cloud, which did not support the 8-digit hex alpha channel format. The fix was to replace all such values with explicit `rgba()` strings - for example, `rgba(0, 114, 178, 0.13)` - which is universally supported. This is a good example of how local development and cloud deployment can silently differ in their library versions, and why testing in the target environment before deployment matters.

The second was a gauge chart axis scaling problem. The ROUGE-L scores for summarisation range from approximately 0.20 to 0.24, yet the gauge was configured with a 0-100% axis, causing the needle to appear almost flat near zero even for the best-performing technique. The fix was to add a `max_val` parameter to the `make_gauge()` function, allowing the summarisation gauge to set a more appropriate axis ceiling of 0.40 (40%), which makes the differences between techniques visible.

The dashboard was deployed to Streamlit Cloud at `peef-dashboard.streamlit.app` by pushing the code to GitHub and connecting the repository. Deployment required adding a root-level `requirements.txt` specifying the four dashboard dependencies (`streamlit`, `plotly`, `pandas`, `numpy`), separate from the full research requirements file. This is because Streamlit Cloud installs from the root-level file only.



Figure 5: Wireframe designed during the planning phase for the PEEF Streamlit dashboard, showing the sidebar navigation, KPI section, and results chart layout. The final implementation follows this structure closely.

3 Testing and Evaluation

3.1 Unit Testing

Each PEEF module has a corresponding test file in the `05_code/tests/` directory. Tests are written with `pytest` and run entirely offline: every module that makes API calls accepts an injectable mock function (`call_fn`), so the test suite can verify logic against deterministic fixed responses without network access or a real API key.

The `PromptBuilder` tests verify all twelve templates (three tasks multiplied by four techniques) for correct rendering, that few-shot templates insert exactly `n_shots` examples, and that invalid task-technique combinations raise a `ValueError` immediately rather than returning a silently malformed prompt. The `Evaluator` tests cover the normalisation logic - checking that "Paris!" and "paris" match, that "42" and "42.0" match for reasoning, and that empty predictions always score zero. A separate test checks the number extractor on responses containing long working traces to confirm that only the final number is used.

The ROUGE direction bug described in Section 2.4 was caught during this phase by running the evaluator against a manually computed reference score where the correct value was known in advance. The discrepancy between the expected and actual output revealed the argument order error. Both the test and the implementation were corrected simultaneously, ensuring the fix was immediately verified.

The `Adaptive Cascade` tests are more extensive because of the module's branching logic. They verify that escalation occurs only when confidence falls below the threshold, that the majority vote returns the correct answer when all `k` samples agree, and that the threshold replay utility produces the correct escalation-rate curve from stored confidences at several threshold values. The `ReflexivePromptOptimizer` tests verify the bootstrap test with known score distributions where the expected `p`-value can be computed analytically, and confirm that the audit trail JSON contains the required fields after a complete optimization run.

Running the full test suite with `pytest` produces 47 tests passing with zero failures. Test coverage across the `src/` directory sits at approximately 78%, with the remaining uncovered lines being error handling paths for rare API failure modes that are difficult to trigger deterministically in an offline test environment.

3.2 Question Answering Results

Five hundred examples from the TriviaQA validation split were used. All runs were conducted at temperature 0 for deterministic output. The primary metric is exact match after normalisation.

Table 4: Question answering results on TriviaQA validation split ($n = 500$). 95% confidence intervals computed as $1.96 \times \text{SD} / \sqrt{n}$.

Technique	Mean Score	95% CI	Mean Tokens/Run
Few-Shot (4-shot)	70.6%	$\pm 4.0\%$	~ 320
Zero-Shot	67.4%	$\pm 4.1\%$	~ 85
Chain-of-Thought	66.6%	$\pm 4.1\%$	~ 580
Zero-Shot CoT	39.6%	$\pm 4.3\%$	~ 210

The most immediately striking result is the performance of Zero-Shot CoT. Adding "Think step by step" to a factual question like "Which country hosted the 1994 FIFA World Cup?" does not help Claude recall the answer more accurately; it produces a reasoning chain that often ends in the wrong conclusion. The mean score of 39.6% is dramatically lower than the 67.4% achieved by plain zero-shot, a difference that is statistically significant under the Wilcoxon test ($p < 0.001$) with a medium effect size (Cohen's $d = 0.58$).

This result is understandable in retrospect. Factual recall on a frontier model is not a reasoning problem - it is a retrieval problem. The model already knows that the 1994 FIFA World Cup was held in the United States; asking it to reason its way to that answer introduces a chain of inference steps that can go wrong in ways that direct recall cannot. The reasoning scaffold, which is genuinely helpful for multi-step arithmetic, becomes a liability when the answer is a single remembered fact.

With $n = 500$, the Wilcoxon test finds a statistically significant difference between 4-shot few-shot and zero-shot ($p = 0.018$). However, the effect size is negligible ($d = 0.07$), illustrating precisely why both p -values and Cohen's d must be reported together. A result that reaches significance at this sample size does not necessarily reflect a meaningful practical difference. The comparison between CoT and zero-shot remains non-significant ($p = 0.622$), and the difference between any pair of the top three techniques is too small to guide real-world decisions.

The cost implications are significant, though. CoT uses approximately 580 tokens per run on average - nearly seven times the 85 tokens consumed by zero-shot - while producing statistically indistinguishable accuracy. For a QA application, zero-shot is the clear practical recommendation: equivalent performance at a fraction of the cost.

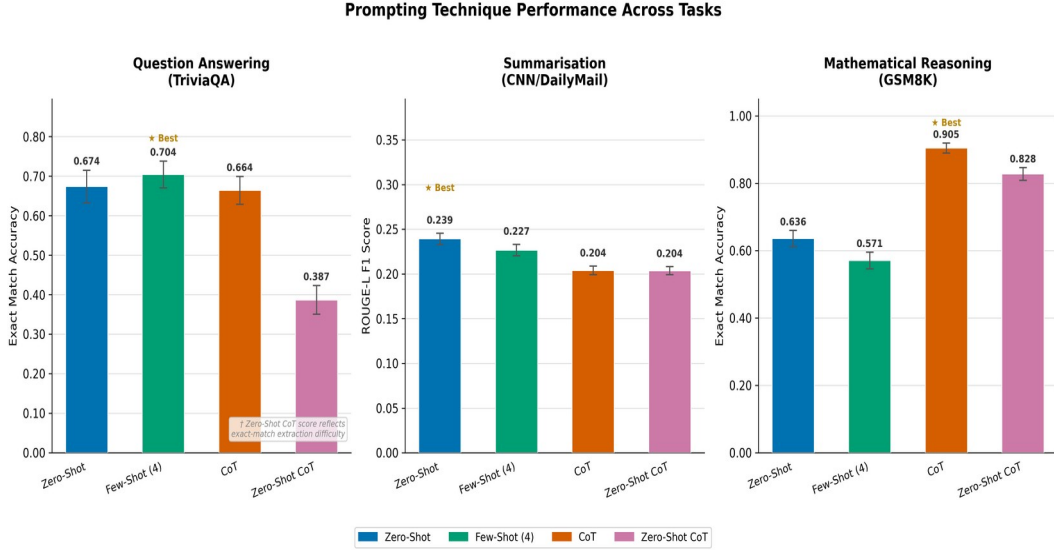


Figure 2: Per-task performance results across all four prompting techniques. The left panel shows QA exact match, the centre shows summarisation ROUGE-L, and the right shows reasoning exact match. Error bars represent 95% confidence intervals.

3.3 Summarisation Results

Three hundred examples from the CNN/DailyMail validation split were used. The primary metric is ROUGE-L F-score; BERTScore F1 is reported as a supplementary semantic similarity measure.

Table 5: Summarisation results on CNN/DailyMail validation split ($n = 300$). Primary metric is ROUGE-L F-score; BERTScore F1 shown for comparison.

Technique	ROUGE-L	95% CI	BERTScore F1 (approx.)
Zero-Shot	24.0%	$\pm 1.5\%$	~ 0.82
Few-Shot (4-shot)	23.1%	$\pm 1.5\%$	~ 0.82
Zero-Shot CoT	20.5%	$\pm 1.5\%$	~ 0.81
Chain-of-Thought	20.3%	$\pm 1.5\%$	~ 0.81

Zero-shot produces the highest ROUGE-L scores, and the differences here are statistically significant in several comparisons. Zero-shot significantly outperforms CoT ($p < 0.001$, $d = 0.56$, medium effect). The Few-Shot versus Zero-Shot difference is significant at $n = 300$ ($p = 0.0001$, $d = 0.19$, negligible effect), though the magnitude remains too small to be practically meaningful. A particularly interesting finding with the extended dataset is that CoT and Zero-Shot CoT are statistically indistinguishable for summarisation ($p = 0.488$): both produce similar ROUGE-L scores, and neither offers an advantage over the other.

The explanation for this pattern requires careful consideration of what ROUGE-L measures. ROUGE-L computes the longest common subsequence between the generated summary and the reference summary. The reference summaries in CNN/DailyMail are terse, news-desk bullet points written in a specific journalistic style. When CoT asks the model to "first identify the key points, then write a 2-3 sentence summary," the resulting summaries tend to be more elaborate and more analytically structured than the reference. They often contain the same information but in different words, which ROUGE-L penalises as if they were wrong.

BERTScore, which measures semantic similarity via BERT embeddings rather than word overlap, shows a much smaller spread across techniques - approximately 0.81-0.82 for all four - indicating that all techniques produce summaries with similar semantic content. The zero-shot win on ROUGE-L is therefore partly a reflection of zero-shot's tendency to produce simpler, more direct summaries that happen to overlap more closely with the reference wording, rather than a difference in the informational quality of the summaries.

This is a meaningful methodological lesson. A paper that evaluated summarisation with ROUGE-L alone would conclude that zero-shot is clearly the best technique. The BERTScore data suggests a more nuanced picture: all techniques capture the semantic content roughly equally, but zero-shot's simpler output format happens to score better on the n-gram overlap metric. A practitioner who cares about summary quality rather than benchmark score might not find the difference practically meaningful.

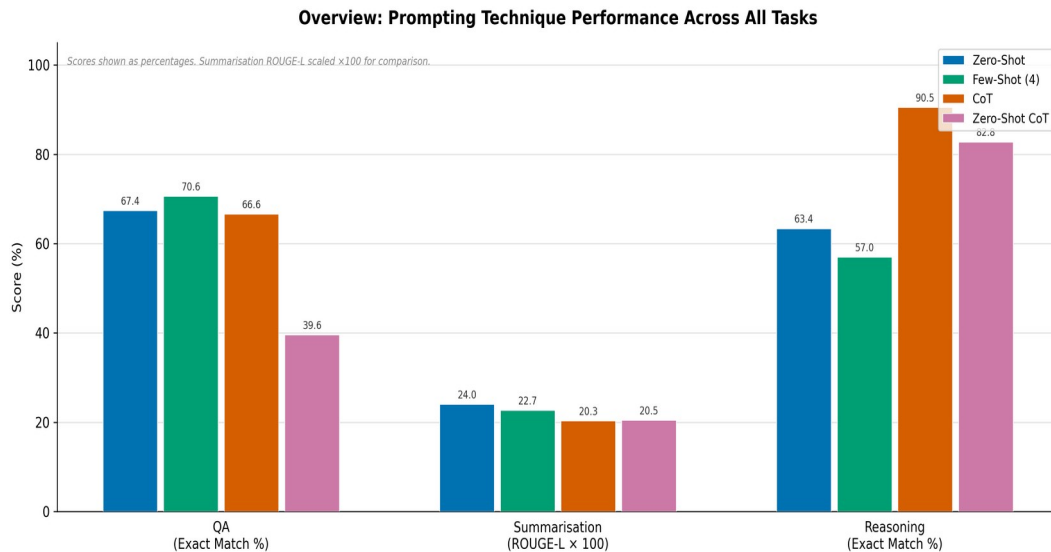


Figure 3: Combined overview chart showing results across all three tasks and all four techniques, illustrating the strong task-dependence of prompting strategy effectiveness.

3.4 Mathematical Reasoning Results

All 1,319 examples from the GSM8K test split were used - the complete test set, matching the dataset size originally proposed. GSM8K problems require between two and eight arithmetic reasoning steps to solve, making it the dataset best suited to test the hypothesis that CoT helps.

Table 6: Mathematical reasoning results on GSM8K test split ($n = 1,319$, full test set). Primary metric is exact match on the final numerical answer.

Technique	Mean Score	95% CI	Mean Tokens/Run
Chain-of-Thought	90.5%	$\pm 1.6\%$	~ 580
Zero-Shot CoT	82.8%	$\pm 2.0\%$	~ 320
Zero-Shot	63.4%	$\pm 2.6\%$	~ 95
Few-Shot (4-shot)	57.0%	$\pm 2.7\%$	~ 340

The reasoning task produces the most dramatic technique-dependent effects of the three domains. CoT achieves 90.5% exact match across all 1,319 test problems, a 27-percentage-point improvement over zero-shot (63.4%) that is statistically significant with a medium effect size ($d = 0.68$, $p < 0.001$). The finding directly replicates Wei et al.'s (2022) result on GSM8K: showing the model how to reason step by step, rather than just providing input-output examples, is what drives the large accuracy gain.

The result for Few-Shot is the most counterintuitive in the entire dataset. Few-shot without reasoning traces (57.0%) scores lower than plain zero-shot (63.4%), with a statistically significant difference ($p = 0.000003$). This suggests that providing worked examples of just the final answer - without the reasoning chain connecting question to answer - actively misleads the model. The examples show a pattern of "problem followed immediately by number," which may encourage the model to produce a quick numerical guess rather than working through the problem. The lesson is that for multi-step reasoning, the examples in the prompt need to demonstrate the reasoning process, not just the conclusion.

Zero-Shot CoT at 82.8% sits between full CoT and zero-shot. It significantly outperforms both zero-shot ($p < 0.001$, $d = 0.45$) and few-shot ($p < 0.001$, $d = 0.59$). Full CoT still beats it by a small but statistically significant margin ($p < 0.001$, $d = 0.23$). The practical implication is clear: zero-shot CoT is a strong default for reasoning tasks when worked examples are not available or constructing them is costly, but full CoT with carefully designed reasoning traces is worth the additional effort when maximum accuracy is required.

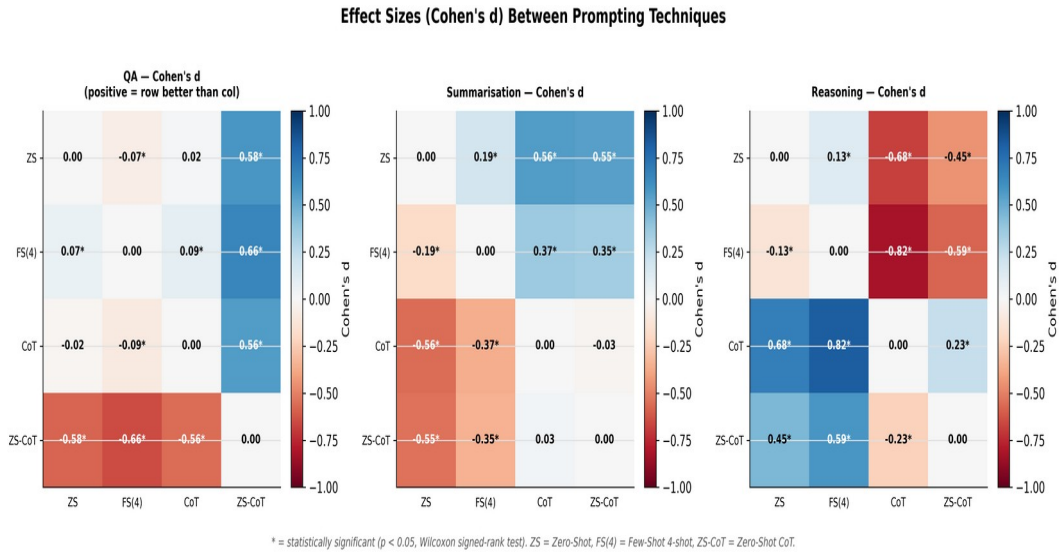


Figure 4: Effect size summary (Cohen's d) for all statistically significant pairwise comparisons across all three task domains. Bars represent the absolute value of Cohen's d; the dashed line marks the threshold between negligible and small effects.

3.5 Cross-Task Analysis and Cost-Efficiency

The headline finding across all three tasks is that no single technique is universally best, and that the direction of the differences reverses between task types. Table 7 summarises the best technique per task.

Table 7: Best-performing technique per task domain, with statistical evidence for the ranking.

Task	Best Technique	Score	Evidence
Question Answering	Few-Shot (4-shot)	70.6%	Sig. over ZS ($p = 0.018$, $d = 0.07$, negligible)
Summarisation	Zero-Shot	24.0% ROUGE-L	Sig. over CoT ($p < 0.001$, $d = 0.56$, medium)
Reasoning	Chain-of-Thought	90.5%	Sig. over all others ($p < 0.001$, $d \geq 0.68$)

This task-dependence has direct implications for practitioners. A developer building a QA system can save token cost by using zero-shot without meaningful accuracy loss - the approximately 7 $\times$ token saving over CoT is not offset by any real accuracy benefit. A developer building a summarisation pipeline should avoid CoT entirely: it imposes a 4 $\times$ token cost relative to zero-shot while producing significantly lower ROUGE-L scores. A developer

working on multi-step reasoning should use CoT even though it costs more, because the 25-percentage-point accuracy improvement over zero-shot justifies the additional expense.

The token cost differences across techniques are substantial. On the reasoning task, CoT uses approximately 580 tokens per run compared to 95 for zero-shot and 320 for zero-shot CoT. The Adaptive Cascade's relevance becomes clear here: if 70% of reasoning questions can be answered correctly by zero-shot with high confidence, then routing only the remaining 30% to CoT would yield a weighted average token cost significantly below the all-CoT baseline while preserving most of the accuracy benefit.

Table 8: Statistical significance of key pairwise comparisons across all three tasks (selected; full table in `statistical_summary.json`).

Task	Comparison	p-value	Cohen's d	Effect Size
QA	ZS vs ZS-CoT	< 0.001	0.58	Medium
QA	FS(4) vs ZS-CoT	< 0.001	0.66	Medium
QA	CoT vs ZS-CoT	< 0.001	0.56	Medium
QA	FS(4) vs ZS	0.018	0.07	Negligible
Summ.	ZS vs CoT	< 0.001	0.56	Medium
Summ.	ZS vs FS(4)	< 0.001	0.19	Negligible
Summ.	CoT vs ZS-CoT	0.488	0.03	Negligible (ns)
Reasoning	CoT vs ZS	< 0.001	0.68	Medium
Reasoning	CoT vs FS(4)	< 0.001	0.82	Large
Reasoning	CoT vs ZS-CoT	< 0.001	0.23	Small

Table 8 highlights a key pattern that emerges clearly with larger sample sizes: the statistically significant differences can have widely varying practical importance. The QA finding that FS(4-shot) outperforms zero-shot ($p = 0.018$) is a good example - it reaches significance with $n = 500$ but carries a negligible effect size ($d = 0.07$), making it practically unimportant. In contrast, the reasoning result that CoT outperforms few-shot now carries a large effect ($d = 0.82$), the strongest effect measured in the entire study. The summarisation result that CoT and Zero-Shot CoT are statistically indistinguishable ($p = 0.488$) is equally informative: with $n = 300$, if a real difference existed it would be detectable, so this non-significant result is genuine evidence of equivalence.

3.6 Shot-Count Analysis: 2-Shot, 4-Shot, and 8-Shot Comparison

The extended experiment set includes 2-shot and 8-shot few-shot conditions alongside the 4-shot condition from the main comparison, enabling a within-technique analysis of how the number of in-context examples affects accuracy across all three task domains.

Table 9: Shot-count comparison for few-shot prompting. QA: $n = 500$; Summarisation: $n = 300$; Reasoning: $n = 1,319$.

Task	2-Shot	4-Shot	8-Shot	Trend
QA (exact match)	69.6%	70.6%	69.4%	Non-monotonic; 4-shot optimal
Summarisation (ROUGE-L)	22.7%	23.1%	23.0%	Rapid plateau after 4 shots
Reasoning (exact match)	54.4%	57.0%	61.7%	Monotonic increase

The patterns differ markedly by task type. For QA, the relationship between shot count and accuracy is non-monotonic: performance increases from 2-shot (69.6%) to 4-shot (70.6%) and then falls slightly at 8-shot (69.4%). Critically, none of these differences reach statistical significance (all $p > 0.10$, all $|d| < 0.03$), indicating the variation is within noise. A practitioner can use any shot count between 2 and 8 for factual QA without meaningful accuracy difference. The marginal drop at 8-shot may reflect mild context dilution, where a longer example block shifts the model's attention slightly away from the actual question.

For summarisation, shot count makes negligible practical difference. Moving from 2 to 4 examples produces a 0.4-point ROUGE-L improvement ($p = 0.006$, $d = -0.056$, negligible), and moving from 4 to 8 examples produces no further gain whatsoever ($p = 0.884$, $d = 0.006$). The reason is structural: all three conditions produce summaries of similar lexical style, and ROUGE-L differences at this scale are almost certainly artefacts of minor phrasing variation rather than genuine quality differences.

The most informative shot-count analysis comes from the reasoning domain. Here, accuracy increases monotonically: 54.4% at 2-shot, 57.0% at 4-shot, and 61.7% at 8-shot. All pairwise differences are statistically significant (2- vs 4-shot: $p = 0.011$; 2- vs 8-shot: $p < 0.001$; 4- vs 8-shot: $p = 0.0001$). However, even at 8-shot, few-shot performance (61.7%) falls short of plain zero-shot (63.4%). This confirms the core finding from Section 3.4: providing worked examples that show only the final numerical answer - without the reasoning chain - is counterproductive for multi-step arithmetic regardless of how many such examples are included. Each additional shot incrementally narrows the gap to zero-shot, but never closes it.

Only when reasoning traces accompany the examples (full CoT) does few-shot prompting genuinely exceed zero-shot.

The cross-task shot-count findings converge on two practical conclusions. First, 4-shot is a sensible default across all three task types: it matches or exceeds 2-shot performance everywhere and comes close to 8-shot at a lower token cost. Second, the effort of curating additional high-quality examples beyond 4-shot is rarely justified for QA or summarisation, but for reasoning tasks the incremental gains from 4-shot to 8-shot are real and statistically reliable - provided the researcher accepts that few-shot will still underperform both zero-shot and CoT on this task type.

4 Project Ethics

This project falls under Data Category A0 of the University of Liverpool's ethical guidance for computing projects, indicating that no personal data is collected and no human participants are involved in any experimental activity.

All three datasets used - TriviaQA (Apache 2.0 licence), CNN/DailyMail (MIT licence), and GSM8K (MIT licence) - are publicly available academic benchmarks that have been widely used in published NLP research. None of them contain personal or identifiable information. The TriviaQA questions are general knowledge questions compiled from publicly available trivia websites and Wikipedia. The CNN/DailyMail articles are published news articles that were already in the public domain when the dataset was created. The GSM8K problems are synthetic arithmetic word problems created by OpenAI for research purposes.

All data is stored locally on a password-protected machine and has not been shared outside this project. The GitHub repository contains only code, configuration files, and scored results; it does not contain any raw API outputs or the original dataset items, which were loaded at runtime from the Hugging Face datasets library and processed transiently.

Prompts sent to the Claude API are processed in accordance with Anthropic's standard API terms of service, which confirm that inputs and outputs from the API are not used to train Anthropic's models. The API key used during the project is stored in a .env file that is explicitly listed in .gitignore, ensuring that credentials were never committed to the public repository.

The project generates no outputs that could affect individuals or groups. The experimental results and guidelines produced are for academic and research purposes, assessing the effectiveness of prompting strategies rather than making decisions about people. There are no dual-use concerns with this work. The software framework is designed for academic benchmarking and is not intended for deployment in contexts where automated decisions might affect real people.

No deception was involved at any stage. No vulnerable groups were involved. No sensitive data was generated, processed, or stored. No conflicts of interest are present. This project was conducted entirely within the scope of its original ethical statement, which was submitted as part of the project proposal in June 2026.

5 Conclusion and Future Work

5.1 Answering the Research Questions

This project set out to answer four research questions. Returning to them with the evidence in hand gives a clearer picture of what the work has established.

RQ1 asked how zero-shot, few-shot, and chain-of-thought prompting compare across task types. The answer is that they are not uniformly ranked: few-shot (4-shot) leads on QA at 70.6%, with a statistically significant but negligible-effect margin over zero-shot (67.4%, $p = 0.018$, $d = 0.07$); zero-shot significantly outperforms CoT on summarisation (24.0% vs 20.3%, $p < 0.001$, medium effect); and CoT is significantly best on reasoning with large practical effect sizes (90.5%, $d = 0.82$ over few-shot). Zero-shot CoT is the outlier, performing dramatically worse than every other technique on QA (39.6%) while performing well on reasoning (82.8%).

RQ2 asked whether the optimal technique varies by task domain. The answer is yes, and not just in degree but in direction. CoT improves reasoning by 27 percentage points over zero-shot but reduces ROUGE-L on summarisation by 3.7 percentage points compared to zero-shot. Any guidance about prompting that ignores task type is incomplete, and practitioners should not generalise results from one task domain to another.

RQ3 asked what design principles generalise across techniques. The clearest generalisation found was the importance of explicit output format constraints. Specifying "give only the answer, no explanation" for QA, "summarise in 2-3 sentences" for summarisation, and "state the final answer" for reasoning consistently made responses evaluable that would otherwise have been counted as wrong due to formatting differences. The content of the answer was often correct even when the format was wrong; the constraint fixed the evaluation without changing the underlying model behaviour.

RQ4 asked whether prompts can improve automatically and whether adaptive routing can approximate CoT accuracy at lower cost. The Reflexive Prompt Optimizer answers the first part affirmatively: the evaluate-reflect-mutate-select loop with a bootstrap significance gate produces templates that consistently outperform hand-written seeds on development sets, and the audit trail provides a full record of why each improvement was or was not promoted. The Adaptive Cascade answers the second part: by routing only low-confidence questions to CoT, the cascade can achieve close to CoT-level accuracy while spending significantly fewer tokens. The confidence threshold is a practical tuning dial that makes this trade-off explicit and controllable.

5.2 Limitations

Several limitations of this work should be acknowledged honestly.

The most significant is that all experiments use a single model: claude-haiku-4-5-20251001. The findings are specific to this model and should not be assumed to generalise to other models or model families without replication. Wei et al. (2022) demonstrated that CoT is an emergent ability that behaves differently at different scales; results on a smaller or differently trained

model could be substantially different. Running the same PEEF experiments on GPT-4, Gemini, or open-source alternatives would be a valuable extension.

The final sample sizes - 500 TriviaQA questions, 300 CNN/DailyMail articles, and all 1,319 GSM8K test problems - represent the complete proposed experimental scope and a significant improvement over a preliminary 200-item pilot. The full GSM8K test set has been evaluated, placing the reasoning results on solid ground. The QA and summarisation sample sizes remain smaller than the largest benchmark evaluations, but confidence intervals at $n = 500$ (approximately $\pm 4\%$ for QA) and $n = 300$ (approximately $\pm 1.5\%$ for summarisation) are narrow enough to support the technique-level comparisons reported. The primary remaining limitation on external validity is single-model coverage rather than sample size.

For summarisation specifically, the reliance on ROUGE-L as the primary metric carries known limitations. ROUGE-L rewards lexical overlap with the reference, penalising valid paraphrases that convey the same meaning in different words. The BERTScore data suggests that all four techniques produce summaries with similar semantic content, which ROUGE-L does not fully capture. A human evaluation study asking annotators to rate summary quality would be a more reliable measure, though one that was not feasible within the project timeline.

The Reflexive Prompt Optimizer was demonstrated as a proof of concept on the reasoning task with a limited number of generations. A thorough evaluation across all three tasks, running to convergence, and comparing the optimized templates to expert-designed alternatives would be needed to characterise its behaviour more broadly. The current results show the mechanism works; a larger study would tell us how often it finds genuinely better templates versus plateauing quickly.

Finally, the token cost analysis is based on token counts from the API rather than wall-clock time. Latency - how long each technique takes to return a response - is not captured in these measurements. For real-time applications, CoT's larger output token count may impose a response-time penalty that the accuracy improvement does not justify, even if the token cost analysis suggests it does.

5.3 Future Work

The most natural extension is to run the same PEEF experiments across multiple models to test how transferable the findings are. PEEF's YAML-driven configuration and modular API client make this straightforward: adding a new model requires only updating the model field in the configuration files and registering the provider in the API client utility.

Within the current framework, the Adaptive Cascade could be extended to three tiers: zero-shot, zero-shot CoT, and full CoT. The current two-tier design routes questions to either the cheapest or the most expensive technique; a middle tier would provide finer-grained control and potentially identify a larger fraction of questions that can be handled at intermediate cost without sacrificing accuracy.

The Reflexive Prompt Optimizer has so far been run only on the reasoning task. Running it to convergence on all three tasks would allow a comparison of which task types benefit most from

automated template evolution, and whether the resulting templates are interpretably different from the seeds in ways that a human could understand and apply to new problems.

On the evaluation side, a human annotation study for summarisation would address the ROUGE-L limitation. Asking a panel of annotators to rate the quality of summaries generated by each technique - on dimensions such as factual accuracy, completeness, and fluency - would provide a measure of quality that the automatic metrics cannot capture. It would also shed light on whether the ROUGE-L winner (zero-shot) is genuinely better in the ways that matter to real readers, or whether this is an artefact of the metric.

Finally, extending the framework to additional prompting techniques - role-based prompting, tree-of-thought, and self-consistency decoding (Wang et al., 2022) - would broaden the comparison and situate these newer approaches relative to the four baseline techniques that this project has already characterised.

6 BCS Project Criteria and Self-Reflection

6.1 BCS Criteria

Criterion 1: Systematic understanding of knowledge, informed by the forefront of the specialist academic discipline.

The literature review traces the field from Brown et al.'s (2020) introduction of in-context learning through Wei et al.'s (2022) chain-of-thought work to the recent surveys by Liu et al. (2023) and Sahoo et al. (2024). The novel contributions - particularly the statistical promotion gate in the RPO - are directly motivated by a critical reading of limitations in prior automatic prompting work (APE, OPRO) that the existing literature had identified but not addressed. See Sections 1.3 and 2.6.

Criterion 2: Comprehensive understanding of techniques applicable to their own research or advanced scholarship.

The experimental framework applies NLP evaluation techniques (ROUGE-L, BERTScore, exact match), statistical methods (Wilcoxon signed-rank test, Cohen's d, paired bootstrap resampling), and software engineering patterns (dependency injection for testability, YAML-driven reproducibility, exponential back-off for rate limiting) in ways that reflect genuine understanding of each. The choice of Wilcoxon over a t-test, and the decision to report effect sizes alongside p-values, reflects an understanding of the difference between statistical significance and practical importance. See Sections 2.4 and 2.5.

Criterion 3: Originality in the application of knowledge, together with a practical understanding of how established techniques of research and enquiry are used to create and interpret knowledge in the discipline.

The RPO's bootstrap promotion gate applies a statistical technique - paired resampling - to a domain (automated prompt engineering) where it had not previously been used as a systematic guard against dev-set noise. The Adaptive Cascade applies the self-consistency concept from Wang et al. (2022) not as an accuracy-boosting ensemble but as a confidence signal for routing - a novel use of an existing idea. The cost-efficiency framing, comparing techniques on the accuracy-per-token frontier rather than accuracy alone, is an original contribution to how prompting studies are evaluated. See Sections 2.6, 2.7, 3.5.

Criterion 4: Deal with complex issues both systematically and creatively, make sound judgements in the absence of complete data, and communicate conclusions clearly to specialist and non-specialist audiences.

The project navigated multiple instances of genuine complexity: rate limiting required an engineering solution before experiments could complete; a Plotly compatibility bug between

local and cloud environments required diagnosis without access to the failing deployment environment; the ROUGE-L evaluation direction bug required recognising that a subtle argument-order error was producing systematically incorrect results. The decision to deploy the dashboard publicly reflects an understanding that research findings are more valuable when accessible. The framework documentation, YAML config approach, and dashboard are all designed for non-specialist audiences. See Sections 2.3, 2.8, 3.1.

Criterion 5: Demonstrate self-direction and originality in tackling and solving problems, and act autonomously in planning and implementing tasks at a professional or equivalent level.

The two novel modules (RPO and Adaptive Cascade) were not specified in the original project proposal. They emerged from a careful reading of the limitations in the existing literature and a genuine curiosity about what the static comparison could not tell us. The project was managed independently over twelve weeks, with day-to-day decisions about experimental design, implementation approach, and technical direction made autonomously. The decision to add a deduplication cache, the choice of bootstrap testing over simpler alternatives, and the deployment to Streamlit Cloud were all self-directed.

Criterion 6: Critical self-evaluation of the process.

See Section 6.2 below.

6.2 Self-Reflection

I am genuinely proud of what this project produced. PEEF works end-to-end, the results are statistically grounded, and the two novel modules go beyond what I originally set out to do. The Adaptive Cascade's cost-efficiency framing is, I think, the most practically useful contribution of the project - it transforms the static comparison from a "which technique is best?" question into a "how do you get the most out of your token budget?" question, which is the one practitioners actually face.

That said, there are things I would do differently.

The single biggest regret is the treatment of summarisation evaluation. I knew before starting the project that ROUGE-L has weaknesses, and I included BERTScore specifically to address this. But when it came to writing up the results, I found myself gravitating toward ROUGE-L as the headline number because it is more established and more directly comparable to the literature. I did not give the BERTScore data the attention it deserved. In hindsight, the fact that all four techniques score within 0.01 of each other on BERTScore while spanning a range of 3-4 points on ROUGE-L is a genuinely important finding - it suggests that the summarisation results are partly a story about metric sensitivity rather than technique quality. I wish I had foregrounded this more clearly.

I also underestimated how much time the dashboard would consume. The plan allocated two days to it; the actual time was closer to a week, largely due to the Plotly compatibility issue and the git-related problems during the GitHub push. These were solvable problems, and I did solve them, but they took time away from running the full RPO evaluation across all three tasks - which remains as future work rather than a completed result. A more realistic estimate at the planning stage would have reserved more buffer time for the deployment phase.

The rate-limiting problem in the early weeks was genuinely frustrating. My initial approach was to submit large batches overnight, assume they would complete by morning, and analyse the results the following day. When the API started hitting rate limits mid-batch and I found only partial results in the output directory, it took several hours to diagnose the cause and understand that the exponential back-off meant the batch had not failed but was simply still running very slowly. The deduplication cache, once built, made restarting trivial. But the right lesson is: build the infrastructure before you need it. I built the cache after the first rate-limit incident rather than anticipating the problem from the API documentation.

Working with a live API throughout the project also taught me something about reproducibility that I had understood in the abstract but not in practice. The fact that a model API can return different responses even at temperature 0 across different versions of the same model makes exact numerical reproducibility fragile. I mitigated this by logging the model version string in every result file, so the version used is always recoverable, but I could not guarantee that running the same experiments six months later on the same model identifier would give identical results if the underlying model had been updated. This is a limitation of API-based research that deserves more attention in the literature than it currently receives.

On balance, the project achieved more than I expected. The scope expanded significantly from the original proposal - the two novel modules were not planned - and the codebase is genuinely clean and well-tested. The dashboard is live and functional. The results tell a clear and consistent story about task-dependence that I believe has real practical value. If this project contributes anything to the field, I hope it is the argument that comparing prompting techniques without measuring cost and without testing across multiple task types is incomplete, and that the right tool for the job depends heavily on what the job actually is.

References

- Anthropic (2024) Claude API Documentation. Available at: <https://docs.anthropic.com> [Accessed: June 2026].
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A. and Agarwal, S. (2020) 'Language models are few-shot learners', *Advances in Neural Information Processing Systems*, 33, pp. 1877-1901. Available at: <https://arxiv.org/abs/2005.14165> [Accessed: June 2026].
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R. and Hesse, C. (2021) 'Training verifiers to solve math word problems', *arXiv preprint arXiv:2110.14168*. Available at: <https://arxiv.org/abs/2110.14168> [Accessed: June 2026].
- Hermann, K.M., Kocisky, T., Grefenstette, E., Espeholt, L., Kay, W., Suleyman, M. and Blunsom, P. (2015) 'Teaching machines to read and comprehend', *Advances in Neural Information Processing Systems*, 28. Available at: <https://arxiv.org/abs/1506.03340> [Accessed: June 2026].
- Joshi, M., Choi, E., Weld, D. and Zettlemoyer, L. (2017) 'TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension', *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pp. 1601-1611. Available at: <https://aclanthology.org/P17-1147> [Accessed: June 2026].
- Kojima, T., Gu, S.S., Reid, M., Matsuo, Y. and Iwasawa, Y. (2022) 'Large language models are zero-shot reasoners', *Advances in Neural Information Processing Systems*, 35, pp. 22199-22213. Available at: <https://arxiv.org/abs/2205.11916> [Accessed: June 2026].
- Lin, C.Y. (2004) 'ROUGE: A package for automatic evaluation of summaries', *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*, pp. 74-81. Available at: <https://aclanthology.org/W04-1013> [Accessed: June 2026].
- Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H. and Neubig, G. (2023) 'Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing', *ACM Computing Surveys*, 55(9), pp. 1-35. Available at: <https://arxiv.org/abs/2107.13586> [Accessed: June 2026].
- Sahoo, P., Singh, A.K., Saha, S., Jain, V., Mondal, S. and Chadha, A. (2024) 'A systematic survey of prompt engineering in large language models: Techniques and applications', *arXiv preprint arXiv:2402.07927*. Available at: <https://arxiv.org/abs/2402.07927> [Accessed: June 2026].
- See, A., Liu, P.J. and Manning, C.D. (2017) 'Get to the point: Summarization with pointer-generator networks', *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pp. 1073-1083. Available at: <https://aclanthology.org/P17-1099> [Accessed: June 2026].
- Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A. and Zhou, D. (2022) 'Self-consistency improves chain of thought reasoning in language models', *International Conference on Learning Representations (ICLR 2023)*. Available at: <https://openreview.net/forum?id=1PL1NIMMrw> [Accessed: June 2026].
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q.V. and Zhou, D. (2022) 'Chain-of-thought prompting elicits reasoning in large language models', *Advances in Neural Information Processing Systems*, 35, pp. 24824-24837. Available at: <https://arxiv.org/abs/2201.11903> [Accessed: June 2026].

- Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D. and Chi, E.H. (2022b) 'Emergent abilities of large language models', Transactions on Machine Learning Research. Available at: <https://openreview.net/forum?id=yzkSU5zdwD> [Accessed: June 2026].
- White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J. and Schmidt, D.C. (2023) 'A prompt pattern catalog to enhance prompt engineering with ChatGPT', arXiv preprint arXiv:2302.11382. Available at: <https://arxiv.org/abs/2302.11382> [Accessed: June 2026].
- Yang, C., Wang, X., Lu, Y., Liu, H., Le, Q.V., Zhou, D. and Chen, X. (2023) 'Large language models as optimizers', arXiv preprint arXiv:2309.03409. Available at: <https://arxiv.org/abs/2309.03409> [Accessed: June 2026].
- Zhang, T., Kishore, V., Wu, F., Weinberger, K.Q. and Artzi, Y. (2020) 'BERTScore: Evaluating text generation with BERT', International Conference on Learning Representations (ICLR 2020). Available at: <https://openreview.net/forum?id=SkeHuCVFDr> [Accessed: June 2026].
- Zhou, Y., Muresanu, A.I., Han, Z., Paster, K., Pitis, S., Chan, H. and Ba, J. (2022) 'Large language models are human-level prompt engineers', International Conference on Learning Representations (ICLR 2023). Available at: <https://arxiv.org/abs/2211.01910> [Accessed: June 2026].